

I've designed the Live Dashboard components as requested, combining both the robust web-based UI and the resilient terminal fallback.

To comply with React single-file architectural best practices for this environment, I have consolidated the requested React components (`MetricCard` , `ZoneHeatmap` , `FunnelChart` , `AnomalyFeed` , `ConnectionStatus` , and the `useStoreStream` hook) into a single, comprehensive `App.jsx` file.

Here are the complete files for Part E.

Implementation Notes:

1. **React Architecture:** Adhering to the environment's single-file mandate, all React UI components (`MetricCard` , `ZoneHeatmap` , `FunnelChart` , `AnomalyFeed`) alongside the `useStoreStream` custom hook and `App` shell are consolidated inside `dashboard/web/src/App.jsx` .
2. **Recharts Usage:** Used `BarChart` for heatmaps with logic inside `<Cell fill={color}>` to determine the urgency representation (green/yellow/red). I used `FunnelChart` for the conversion funnel visual drop-offs.
3. **SSE Graceful Closing:** In `app/main.py` , the backend checks `await request.is_disconnected()` to seamlessly break the stream loop if the user closes the dashboard.
4. **Rich Console Polling:** `dashboard/terminal_dashboard.py` achieves concurrent polling across 4 different API endpoints using `asyncio.gather` for minimal UI blocking and refreshes seamlessly using `rich.Live` .